

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6
«Работа с БД в СУБД MongoDB»
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся Янченко Денис Сергеевич
Факультет прикладная информатика
Группа K3241
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна
Белов Александр Олегович

Санкт-Петербург
2026

ВВЕДЕНИЕ

Цель работы

Овладеть практическими навыками установки СУБД MongoDB, работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

ВЫПОЛНЕНИЕ

Практическое задание 1:

1. Установите MongoDB для обеих типов систем (32/64 бита).
2. Проверьте работоспособность системы запуском клиента mongo.
3. Выполните методы:
 - a) db.help()

```
test> db.help()

Database Class:

getMongo           Returns the current database connection
getName            Returns the name of the DB
getCollectionNames Returns an array containing the names of all collections in the current database.
getCollectionInfos Returns an array of documents with collection information, i.e. collection name and options, for the current database.
runCommand         Runs an arbitrary command on the database.
adminCommand       Runs an arbitrary command against the admin database.
aggregate          Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
getSiblingDB       Returns another database without modifying the db variable in the shell environment.
getCollection       Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
dropDatabase        Removes the current database, deleting the associated data files.
createUser          Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already exists on the database.
updateUser          Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
changeUserPassword  Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
logout             Ends the current authentication session. This function has no effect if the current session is not authenticated.
dropUser           Removes the user from the current database.
dropAllUsers        Removes all users from the current database.
auth               Allows a user to authenticate to the database from within the shell.
grantRolesToUser    Grants additional roles to a user.
revokeRolesFromUser Removes a one or more roles from a user on the current database.
getUsers            Returns information for a specified user. Run this method on the user's database. The user must exist on the database on which the method runs.
getCollections       Returns information for all the users in the database.
createCollection     Create new collection
createEncryptedCollection Creates a new collection with a list of encrypted fields each with unique and auto-created data encryption keys (DEKs). This is a utility function that internally utilizes ClientEncryption.createEncryptedCollection.
createView           Create new view
createRole           Creates a new role.
updateRole           Updates the role's profile on the database on which you run the method. An update to a field completely replaces the previous field's values.
dropRole            Removes the role from the current database.
dropAllRoles        Removes all roles from the current database.
grantRolesToRole     Grants additional roles to a role.
revokeRolesFromRole Removes a one or more roles from a role on the current database.
grantPrivilegesToRole Grants additional privileges to a role.
revokePrivilegesFromRole Removes a one or more privileges from a role on the current database.
getRole             Returns role information for a specified role. Run this method on the role's database. The role must exist on the database on which the method runs.
getRoles            Returns information for all the roles in the database.
currentOp           Runs an aggregation using $currentOp operator. Returns a document that contains information on in-progress operations for the database instance. For further information, see $currentOp.
killOp              Calls the killOp command. Terminates an operation as specified by the operation ID. To find operations and their corresponding IDs, see $currentOp or db.currentOp().
shutdownServer      Calls the shutdown command. Shuts down the current mongod or mongos process cleanly and safely. You must issue the db.shutdownServer() operation against the admin database.
fsyncLock           Calls the fsync command. Forces the mongod to flush all pending write operations to disk and locks the entire mongod instance to prevent additional writes until the user releases the lock with a corresponding db.fsunlock() command.
fsyncUnlock         Calls the fsyncUnlock command. Reduces the lock taken by db.fslock() on a mongod instance by 1.
version            returns the db version. uses the buildinfo command
serverBits         returns the db serverBits. uses the buildinfo command
isMaster            Calls the isMaster command
hello              Calls the hello command
serverBuildInfo     returns the db serverBuildInfo. uses the buildinfo command
serverStatus        returns the server status. uses the serverStatus command
stats              returns the db stats. uses the dbStats command
hostInfo            Calls the hostInfo command
serverCmdLineOpts   returns the db serverCmdLineOpts. uses the getCmdLineOpts command
rotateCertificates  Calls the rotateCertificates command
printCollectionStats Prints the collection.stats for each collection in the db.
getProfilingStatus  returns the db getProfilingStatus. uses the profile command
setProfilingLevel   returns the db setProfilingLevel. uses the profile command
setLogLevel         returns the db setLogLevel. uses the setParameter command
getLogComponents    returns the db getLogComponents. uses the getParameter command
cloneDatabase        deprecated, non-functional
cloneCollection      deprecated, non-functional
copyDatabase         deprecated, non-functional
commandHelp         returns the db commandHelp. uses the passed in command with help: true
listCommands        Calls the listCommands command
getLastErrorObj      Calls the getLastError command
getLastError         Calls the getLastError command
printShardingStatus  Calls sh.status(verbose)
printSecondaryReplicationInfo Prints secondary replicaset information
getReplicationInfo  Returns replication information
printReplicationInfo Formats sh.getReplicationInfo
printSlaveReplicationInfo DEPRECATED. Use db.printSecondaryReplicationInfo
setSecondaryOk       This method is deprecated. Use db.getMongo().setReadPref() instead
watch               Opens a change stream cursor on the database
sql                 (Experimental) Runs a SQL query against Atlas Data Lake. Note: this is an experimental feature that may be subject to change in future releases.
checkMetadataConsistency Returns a cursor with information about metadata inconsistencies
```

- b) db.stats()

```
test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
```

4. Создайте БД learn.

```
test> use learn
switched to db learn
```

5. Получите список доступных БД.

```
learn> show dbs
admin    40.00 KiB
config   60.00 KiB
local    40.00 KiB
```

6. Создайте коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.

```
learn> db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a186b5f78b44340b1abc114') }
}
```

7. Просмотрите список текущих коллекций.

```
learn> show collections
unicorns
```

8. Переименуйте коллекцию unicorns.

```
learn> db.unicorns.renameCollection("updated_unicorns")
{ ok: 1 }
```

9. Просмотрите статистику коллекции.

```
learn> db.updated_unicorns.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersion=1),assert=(commit_timestamp=none,timestamp=none,write_timestamp=off),block_allocation=best,block_compressor=snappy,block_manager=default,cache_resident=false,checksum=true,columns=,dictionary=0,disaggregated=(page_log=,storage_tier=none),encryption=(keyid=,name=),exclusive=false,extractor=man_value=,ignore_in_memory_cache_size=false,immutable=false,import=(compare_timestamp=oldest_timestamp,enabled=false,file_corrupt=true,repair=false),in_memory=false,ingest=,internal_item_max=0,internal_key_max=0,internal_key_truncate=true,internal_page_gap=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_max=64MB,log=(enabled=true),lsm=(auto_throttle=,bloom_hash_count=,bloom_oldest=,chunk_count_limit=,chunk_max=,chunk_size=,merge_custom=(prefix=,start_generation=,suffix=),page_image_max=0,memory_page_max=10M,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,prefix_compression_min=4096,split_deepen_per_child=0,split_pct=90,stable=,tiered_storage=(auth_token=,bucket=,bucket_prefix=,cache_directory=,local_retention=0),type=file,value_format=u,verbose=[],write_timestamp_usage=none',
    type: 'file',
    uri: 'statistics:table:collection-2b9b7b2f-7754-4ceb-8c08-1e4b85dc81a3',
    version: 'WiredTiger 12.0.0: (November 15, 2024)',
    autocommmit: {
      'retries for readonly operations': 0,
      'retries for update operations': 0
    },
    backup: {
      'total modified incremental blocks with compressed data': 0,
      'total modified incremental blocks without compressed data': 0
    },
    'block-disagg': {
      'Bytes read from the shared history store in SLS': 0,
      'Bytes written to the shared history store in SLS': 0,
      'Disaggregated block manager get': 0,
      'Disaggregated block manager get cold page': 0,
      'Disaggregated block manager get from the shared history store in SLS': 0,
      'Disaggregated block manager page discard calls': 0,
      'Disaggregated block manager put': 0,
      'Disaggregated block manager put cold page': 0,
      'Disaggregated block manager put to the shared history store in SLS': 0,
      'Disaggregated block manager read ahead of materialization frontier': 0
    },
    'block-manager': {
      'allocations requiring file extension': 4,
      'blocks allocated': 4,

```

10. Удалите коллекцию.

```
learn> db.updated_unicorns.drop()
true
```

11. Удалите БД learn.

```
learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
```

Практическое задание 2.1.1:

1) *Создайте базу данных learn.*

2) *Заполните коллекцию единорогов unicorns:*

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm',
vampires: 63});

db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f',
vampires: 43});

db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm',
vampires: 182});

db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires:
99});

db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550,
gender:'f', vampires:80});

db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f',
vampires: 40});

db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm',
vampires: 39});

db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm',
vampires: 2});

db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f',
vampires: 33});

db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm',
vampires: 54});

db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

3) *Используя второй способ, вставьте в коллекцию единорогов документ:*

```
{name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
```

4) *Проверьте содержимое коллекции с помощью метода find.*

```
test> use learn
switched to db learn
learn> db.unicorns.insertMany([
| {name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63},
| {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
| {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
| {name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
| {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80},
| {name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
| {name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
| {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
| {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
| {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
| {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a186d88bc5cb3ff03abc114'),
    '1': ObjectId('6a186d88bc5cb3ff03abc115'),
    '2': ObjectId('6a186d88bc5cb3ff03abc116'),
    '3': ObjectId('6a186d88bc5cb3ff03abc117'),
    '4': ObjectId('6a186d88bc5cb3ff03abc118'),
    '5': ObjectId('6a186d88bc5cb3ff03abc119'),
    '6': ObjectId('6a186d88bc5cb3ff03abc11a'),
    '7': ObjectId('6a186d88bc5cb3ff03abc11b'),
    '8': ObjectId('6a186d88bc5cb3ff03abc11c'),
    '9': ObjectId('6a186d88bc5cb3ff03abc11d'),
    '10': ObjectId('6a186d88bc5cb3ff03abc11e')
  }
}
```

```
learn> document = ({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165})
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a186e13bc5cb3ff03abc11f')
}
```

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],

```

Практическое задание 2.2.1:

1) Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a186e13bc5cb3ff03abc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

2) Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a186d88bc5cb3ff03abc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc114'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc116'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc117'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11a'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11b'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11d'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a186e13bc5cb3ff03abc11f'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]
```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления


```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a186e13bc5cb3ff03abc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  }
]
```

Практическое задание 2.1.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
```

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

Практическое задание 3.1.1:

1. *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",
populatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: [""],
mayor: {
  name: "Jim Wehrle"
}}

{name: "New York",
populatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
party: "I"}}

{name: "Portland",
populatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
party: "D"}}
```

2. *Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I") . Вывести только название города и информацию о мэре.*

3. *Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует) . Вывести только название города и информацию о мэре.*

```

learn> db.towns.insertMany([
  {name: "Punxsutawney ",
    populatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: [""],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {name: "New York",
    populatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {name: "Portland",
    populatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a1872c3bc5cb3ff03abc120'),
    '1': ObjectId('6a1872c3bc5cb3ff03abc121'),
    '2': ObjectId('6a1872c3bc5cb3ff03abc122')
  }
}
learn> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn> db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
[ { name: 'Punxsutawney ', mayor: { name: 'Jim Wehrle' } } ]

```

Практическое задание 3.1.2:

- 1) Сформировать функцию для вывода списка самцов единорогов.
- 2) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
- 3) Вывести результат, используя *forEach*.
- 4) Содержание коллекции единорогов *unicorns*:

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600,
gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450,
gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight:
984, gender: 'm', vampires: 182});
```

```
db.unicorns.insert({name: 'Rooooooodles', 44), loves: ['apple'], weight: 575,
gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'],
weight:550, gender:'f', vampires:80});
```

```
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733,
gender: 'f', vampires: 40});
```

```
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690,
gender: 'm', vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421,
gender: 'm', vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
```

```
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
```

```
db.unicorns.insert ({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

```
db.unicorns.insert ({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165})
```

```
learn> fn = function() { return this.gender == "m"; }  
[Function: fn]  
learn> var cursor = db.unicorns.find({$where: fn}).sort({name: 1}).limit(2);  
  
learn> cursor.forEach(function(obj){ print(obj.name); })  
Dunx  
Horny
```

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()  
2
```

Практическое задание 3.2.2:

Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves")  
[  
  'apple',      'carrot',  
  'chocolate', 'energon',  
  'grape',      'lemon',  
  'papaya',     'redbull',  
  'strawberry', 'sugar',  
  'watermelon'  
]
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}]  
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
```

Практическое задание 3.3.1:

1. *Выполнить команду:*

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],  
weight: 340, gender: 'm'})
```

2. *Проверить содержимое коллекции unicorns.*

```

learn> db.unicorns.save({name: 'Barney', loves: ['grape'],
| weight: 340, gender: 'm'})
TypeError: db.unicorns.save is not a function
learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'],
| weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a187505bc5cb3ff03abc123')
}
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {

```

Практическое задание 3.3.2:

1. Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
2. Проверить содержимое коллекции `unicorns`.

```

learn> db.unicorns.update({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]

```

Практическое задание 3.3.3:

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.update({name: 'Raleigh'}, {$push: {loves: 'redbull'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Практическое задание 3.3.4:

1. *Всем самцам единорогов увеличить количество убитых вампиров на 5.*
2. *Проверить содержимое коллекции unicorns.*

```
learn> db.unicorns.update({gender: 'm'}, {$inc: {vampires: 5}}, {multi: true})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  }
]
```

Практическое задание 3.3.5:

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

2. Проверить содержимое коллекции towns.

```
learn> db.towns.update({name: 'Portland'}, {$unset: {"mayor.party": 1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a1872c3bc5cb3ff03abc122'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Практическое задание 3.3.6:

1. Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.

2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.update({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

Практическое задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.

2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.update({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```


Практическое задание 3.4.1:

4. *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",
popujatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: ["phil the groundhog"],
mayor: {
  name: "Jim Wehrle"
}}

{name: "New York",
popujatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
party: "I"}}

{name: "Portland",
popujatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
party: "D"}}
```

5. *Удалите документы с беспартийными мэрами.*

6. *Проверьте содержание коллекции.*

7. *Очистите коллекцию.*

8. *Просмотрите список доступных коллекций.*

```
learn> db.towns.insertMany([ { name: "Punxsutawney ", populatiuon: 6200, last_sensus: ISODate("2008-01-31"), famous_for: [""],
mayor: { name: "Jim Wehrle" } }, { name: "New York", populatiuon: 22200000, last_sensus: ISODate("2009-07-31"), famous_for: [
"status of liberty", "food"], mayor: { name: "Michael Bloomberg", party: "I" } }, { name: "Portland", populatiuon: 528000, las
t_sensus: ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: { name: "Sam Adams", party: "D" } } ] )
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a187778bc5cb3ff03abc127'),
    '1': ObjectId('6a187778bc5cb3ff03abc128'),
    '2': ObjectId('6a187778bc5cb3ff03abc129')
  }
}
learn> db.towns.deleteMany({mayor.party: {$exists: false}})
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a187778bc5cb3ff03abc128'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a187778bc5cb3ff03abc129'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
```

Практическое задание 4.1.1:

1) *Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.*

2) *Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.*

3) *Проверьте содержание коллекции едиорогов.*

4) *Содержание коллекции единорогов unicorns:*

```

db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600,
gender: 'm', vampires: 63});

db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450,
gender: 'f', vampires: 43});

db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight:
984, gender: 'm', vampires: 182});

db.unicorns.insert({name: 'Roooooodles', 44), loves: ['apple'], weight: 575,
gender: 'm', vampires: 99});

db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'],
weight:550, gender:'f', vampires:80});

db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733,
gender: 'f', vampires: 40});

db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690,
gender: 'm', vampires: 39});

db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421,
gender: 'm', vampires: 2});

db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601,
gender: 'f', vampires: 33});

db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650,
gender: 'm', vampires: 54});

db.unicorns.insert ({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540,
gender: 'f'});

db.unicorns.insert {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704,
gender: 'm', vampires: 165}

```

```

learn> db.zones.insertMany([
| {_id: "mf", name: "Mysterious Forest", description: "A dense, enchanted woodland covered in perpe
tual mist and glowing ancient trees."},
| {_id: "fi", name: "Flying Island", description: "A majestic landmass floating high in the clouds
with waterfalls descending into the open sky."}
| ])
{ acknowledged: true, insertedIds: { '0': 'mf', '1': 'fi' } }
learn> db.unicorns.update({name: 'Horny'}, {$set: {zone: {$ref: "zones", $id: "mf"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update({name: 'Aurora'}, {$set: {zone: {$ref: "zones", $id: "fi"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update({name: 'Unicrom'}, {$set: {zone: {$ref: "zones", $id: "mf"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68,
    zone: DBRef('zones', 'mf')
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
    zone: DBRef('zones', 'fi')
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187,
    zone: DBRef('zones', 'mf')
  },
  {
    _id: ObjectId('6a186d88bc5cb3ff03abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575
  }
]
```

Практическое задание 4.2.1:

1. Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

2. Содержание коллекции единорогов `unicorns`:

```
db.unicorns.insert({name: 'Horny', dob: new Date(1992,2,13,7,47), loves:
['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', dob: new Date(1991, 0, 24, 13, 0), loves:
['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', dob: new Date(1973, 1, 9, 22, 10), loves:
['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
```

```
db.unicorns.insert({name: 'Rooooooodles', dob: new Date(1979, 7, 18, 18, 44),
loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', dob: new Date(1985, 6, 4, 2, 1),
loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
```

```
db.unicorns.insert({name:'Ayna', dob: new Date(1998, 2, 7, 8, 30), loves:
['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
```

```
db.unicorns.insert({name:'Kenny', dob: new Date(1997, 6, 1, 10, 42), loves:
['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', dob: new Date(2005, 4, 3, 0, 57), loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', dob: new Date(2001, 9, 8, 14, 53), loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
```

```
db.unicorns.insert({name: 'Pilot', dob: new Date(1997, 2, 1, 5, 3), loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
```

```
db.unicorns.insert({name: 'Nimue', dob: new Date(1999, 11, 20, 16, 15), loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

```
db.unicorns.insert({name: 'Dunx', dob: new Date(1976, 6, 18, 18, 18), loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165
```

```
learn> db.unicorns.ensureIndex({"name": 1}, {"unique": true})
[ 'name_1' ]
```

Практическое задание 4.3.1:

- 1) Получите информацию о всех индексах коллекции *unicorns*.
- 2) Удалите все индексы, кроме индекса для идентификатора.
- 3) Попробуйте удалить индекс для идентификатора.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
```

Практическое задание 4.4.1:

- 1) Создайте объемную коллекцию *numbers*, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```

- 2) Выберите последних четыре документа.
- 3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра *executionTimeMillis*)
- 4) Создайте индекс для ключа *value*.
- 5) Получите информацию о всех индексах коллекции *numbers*.
- 6) Выполните запрос 2.
- 7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
- 8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

```

learn> for(i = 0; i < 100000; i++){db.numbers.insertOne({value: i})}
{
  acknowledged: true,
  insertedId: ObjectId('6a187aabbcb5cb3ff03ad8fa3')
}
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a187aabbcb5cb3ff03ad8fa3'), value: 99999 },
  { _id: ObjectId('6a187aabbcb5cb3ff03ad8fa2'), value: 99998 },
  { _id: ObjectId('6a187aabbcb5cb3ff03ad8fa1'), value: 99997 },
  { _id: ObjectId('6a187aabbcb5cb3ff03ad8fa0'), value: 99996 }
]
learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', nss: 'learn.numbers', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 101,

```

```

learn> db.numbers.ensureIndex({"value": 1})
[ 'value_1' ]
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a187aabb5cb3ff03ad8fa3'), value: 99999 },
  { _id: ObjectId('6a187aabb5cb3ff03ad8fa2'), value: 99998 },
  { _id: ObjectId('6a187aabb5cb3ff03ad8fa1'), value: 99997 },
  { _id: ObjectId('6a187aabb5cb3ff03ad8fa0'), value: 99996 }
]
learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        nss: 'learn.numbers',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'learn.numbers',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {

```

- До создания индекса: При поиске использовалось полное сканирование коллекции. Время выполнения (executionTimeMillis) составило 101 мс.
- После создания индекса: При поиске использовалось сканирование по индексу. Время выполнения (executionTimeMillis) составило 0 мс.

Запрос с установленным индексом намного эффективнее. Индекс позволил базе данных не перебирать все 100 000 документов, а сразу взять 4 нужных значения, сократив время поиска практически до нуля.

ВЫВОД

В ходе выполнения лабораторных работ были успешно освоены базовые механизмы нереляционной СУБД MongoDB: установка, настройка, создание БД и коллекций. Получены практические навыки написания CRUD-запросов, построения сложных выборок с использованием операторов поиска и фильтрации массива. Изучены механизмы агрегирования, связывания документов (DBRef) и влияние индексов на производительность запросов при больших объемах данных.